

VANI

Voice Analysis & Neural Intelligence System

Presentation Companion & Glossary

A plain-language guide to every concept and term in the VANI presentation

This document accompanies the VANI slide deck. It walks through the presentation section by section, explains how to read each results chart, and defines every technical term used in the slides — assuming no prior background in speech recognition.

Date: 21 July 2026
M.Tech Research Project · IIT Indore

1. The whole project in one paragraph

VANI is a computer system that listens to foreign-language military radio conversations and automatically produces an English intelligence summary — running entirely offline on a single laptop, with no internet. It does this in a chain of steps: detect where speech is, identify which language is being spoken, transcribe that speech into text, translate the text to English, flag dangerous keywords, and finally write a short structured summary. The research core of the project was choosing the *best speech-recognition engine for each language*. The team first fine-tuned OpenAI's Whisper model for seven border-region languages, but a careful, corrected evaluation showed a different model — Meta's **SeamlessM4T** — was actually more accurate, and stayed more accurate under noisy radio conditions. After a campaign of targeted improvements, **all seven languages now run on SeamlessM4T**, and the fine-tuned Whisper models are kept only as a backup. The headline result: SeamlessM4T beats fine-tuned Whisper on word-error rate for every language.

The rest of this document explains each part of that story and defines every term.

2. Walkthrough of the presentation

The deck has seven sections. Here is what each one covers and why it matters.

2.1 Introduction & Problem Statement

Sets up the operational problem. Analysts face a large volume of radio traffic in many low-resource border languages, and the system must run **fully offline** (air-gapped) for security. Off-the-shelf speech recognition performs poorly on these languages and on the noisy, band-limited radio channel, and manual transcription cannot keep up. The goal is one end-to-end offline pipeline that turns raw audio into actionable intelligence (who / what / where / threat), not just a raw transcript.

2.2 Deliverables

What was actually built and shipped: the working VANI system with a graphical interface; a seven-language speech-recognition backend that is auto-selected by language; the full intelligence pipeline (transcription, translation, threat detection, summary, storage); the backend-selection research and its corrected evaluation; analyst features (map, search, live microphone, alerts); an optional three-machine networked mode; and the written reports.

2.3 Architecture

The system runs on one offline workstation. It is organised as a stack of models, each doing one job: **Silero VAD** finds speech; **MMS-LID** identifies the language; **SeamlessM4T** transcribes it; **NLLB-200** translates to English; a keyword detector flags threats; and **Gemma 3** (run locally through **Ollama**) writes the summary. Everything is stored in a local database. A key idea is **backend selection**: the system routes each language to whichever speech engine is most accurate for it — today, all seven route to SeamlessM4T (four with a small trained add-on called a LoRA adapter, three used as-is).

2.4 Pipeline

Shows how a single intercept flows through ten sequential stages, from voice-activity detection through to the final database write and report. This is the runtime path every audio clip takes.

2.5 Features

The analyst-facing capabilities: multilingual transcription, language identification, English translation, keyword and threat detection (including a lexicon of coded terminology), the structured intelligence summary, speaker diarization (who spoke when), full-offline operation, GPU acceleration, a searchable database, and the graphical interface with map, dashboard and live-microphone capture.

2.6 Results

The heart of the deck. It presents the central finding — SeamlessM4T beats fine-tuned Whisper on word-error rate — through three charts (explained in Part 3 below), a cross-model comparison table, a robustness evaluation under simulated radio noise, the Kashmiri scoring-ruler correction, and a one-slide summary of how the backend was chosen. It also lists the key findings, including the five scoring-methodology mistakes that were caught and corrected during the project.

2.7 Future Work

Where the system could go next: further accuracy improvements, extending the networked mode, per-speaker transcription, and productisation considerations (including model licensing).

3. How to read the three result charts

3.1 The dumbbell chart (SeamlessM4T vs fine-tuned Whisper)

Each row is one language. Two dots sit on a horizontal word-error-rate axis (lower is better): a **grey** dot for the fine-tuned Whisper model and a **teal** dot for the deployed SeamlessM4T backend. An arrow connects them, pointing to the SeamlessM4T dot — which is always further left (lower error). The green “–X pp” label is the size of the improvement in percentage points. Rows are sorted by improvement, so the biggest win (Punjabi, from 57% down to 20%) is on top. The single takeaway: every arrow points the same way, so SeamlessM4T wins every language.

3.2 The robustness heatmap

A grid of languages (rows) against five radio-channel conditions (columns): clean, telephone bandpass, two noise levels, and MP3 compression. Each cell shows how many percentage points **better** SeamlessM4T is than fine-tuned Whisper under that condition. Darker green means a bigger advantage. Every cell is positive (SeamlessM4T wins everywhere), and several cells get darker in the harsh 0 dB noise column — meaning the advantage *grows* as the channel gets worse, which is exactly what you want for real radio.

3.3 The Kashmiri ruler-correction bars

Three pairs of bars comparing Whisper (grey) and the SeamlessM4T adapter (teal) for Kashmiri, using three different *measuring sticks*. On raw word-error rate (left pair) Whisper looks slightly ahead. But Kashmiri text is written with many small diacritic marks that both systems tend to drop, which unfairly penalises both. Once those marks are normalised away (middle pair) and on character-error rate (right pair), the SeamlessM4T adapter is clearly ahead. The point of the chart: the apparent gap was an artefact of *how the score was measured*, not a real weakness of the model.

4. Glossary — every term in the deck

Grouped by topic. Terms are defined in plain language.

4.1 Accuracy metrics

WER (Word Error Rate)	The main accuracy score for speech recognition: the percentage of words the system got wrong (insertions, deletions, substitutions) versus a human reference transcript. Lower is better. 20% WER means roughly one word in five is wrong.
CER (Character Error Rate)	The same idea as WER but counted per character instead of per word. It is the fairer metric for scripts where word boundaries are ambiguous (Chinese, Perso-Arabic), because a single spelling variation there can wrongly count as a whole wrong word under WER.
chrF	A translation-quality score (character n-gram F-score). Measures how close the machine's English translation is to a reference translation. Higher is better.
pp (percentage points)	The plain difference between two percentages. Going from 20% WER to 12% WER is an improvement of 8 pp (not “8 percent”).
Held-out test / n=100	Accuracy is measured on data the model never saw during training (“held-out”), so the number reflects real generalisation, not memorisation. “n=100” means 100 test clips per language.
Baseline	The starting-point score before any improvement — here, the un-fine-tuned Whisper large-v3 model. Improvements are measured against it.

4.2 Models and tools

ASR (Automatic Speech Recognition)	The technology that converts spoken audio into written text — the core task of the project.
Whisper	OpenAI's open speech-recognition model. Comes in sizes (large-v3, medium, turbo). The project first fine-tuned Whisper for each language; it is now the rollback option.
SeamlessM4T (v2)	Meta's multilingual speech model that both transcribes and translates. The project's corrected evaluation found it more accurate than fine-tuned Whisper, so it is now the deployed engine for all seven languages.
Zero-shot	Using a model as-is, with no additional training for the task or language. SeamlessM4T serves Punjabi, Urdu and Mandarin zero-shot.
MMS-LID / MMS-LID-256	Meta's “Massively Multilingual Speech” language-identification model — it listens to a clip and says which of 256 languages it is. VANI uses it to route each clip to the right speech engine.
NLLB-200	“No Language Left Behind”, Meta's translation model covering 200 languages. VANI uses it to translate transcripts into English.
IndicTrans2	An Indian-language translation model, used specifically for Dogri.
Gemma 3	Google's open large language model, run locally to write the intelligence summary. VANI uses the offline version so no data leaves the machine.

Ollama	A tool for running large language models (like Gemma 3) locally on your own machine, offline. VANI talks to it to generate summaries and to power the chat assistant.
ISUM (Intelligence Summary)	The final structured output: a short “5W” report (who, what, when, where, why) generated from the transcript and translation by the local language model.
Silero VAD	A Voice Activity Detector — it finds the parts of the audio that actually contain speech and discards silence, so the rest of the pipeline only processes real speech.
Streamlit	The Python framework used to build VANI's web-style graphical interface.
SQLite	A lightweight, file-based database. VANI stores transcripts, translations and summaries in it so they can be searched later.

4.3 Training methods

Fine-tuning	Taking a general pre-trained model and training it a little more on your specific data so it performs better on your task/language.
LoRA (Low-Rank Adaptation)	An efficient fine-tuning method: instead of retraining all of a model's (billions of) weights, you train a tiny add-on (well under 1% of the parameters) and leave the original frozen. Cheap, fast, and avoids “forgetting”.
Adapter	The small trained add-on that LoRA produces. It can be switched on for one language and off for others, all on top of the same base model.
Rank (r) and alpha (α)	The two main LoRA dials. Rank sets how much capacity the adapter has (r=8, 16, 32 in this project — bigger = more capacity); alpha scales its effect. Kashmiri and Pashto needed the larger r=32 adapters to win.
q/k/v/out_proj, MLP (fc1/fc2)	The specific internal parts of the model that the adapter attaches to (attention projections and the feed-forward layers). Attaching to more of them gives the adapter more room to adapt.
Custom <code>__kas__</code> / <code>< ks ></code> token	Kashmiri is not in the models' built-in vocabulary, so a new “this-is-Kashmiri” marker token was added to the model and its meaning learned during training — the key step that let SeamlessM4T handle Kashmiri at all.
Trainable embedding row (trainable_token_indices)	Normally LoRA cannot change a model's vocabulary table. This technique makes just the one new Kashmiri token's entry trainable, so its “meaning vector” is learned rather than frozen as a copy of Urdu — the change that finally made Kashmiri competitive.
Noise-augmented training	Deliberately degrading the training audio with the same bandpass/noise/codec effects the system is tested on, so the model learns to stay accurate under radio conditions. This is what finally made Pashto win under noise.
Backend selection / routing	Choosing, per language, which speech engine to use — decided by measured accuracy on clean and noisy test data. In VANI this happens at pipeline Stage 3.5.

4.4 Training data

FLEURS	A Google multilingual speech dataset (read sentences in many languages) used as the primary training and test data.
---------------	---

IndicVoices-R	An AI4Bharat dataset of Indian-language speech, added to improve Hindi, Nepali, Punjabi and Kashmiri — and the only available source for Kashmiri.
Common Voice	Mozilla's crowd-sourced multilingual speech dataset, used to add Pashto training data.
AI4Bharat	An Indian research group (IIT Madras) that produces Indian-language datasets and models, including IndicVoices-R and IndicTrans2.

4.5 Audio, radio and signal terms

Radio intercept	A recorded radio transmission captured for analysis — the system's input.
Robustness / degradation conditions	How well accuracy holds up when the audio is damaged. The project tests five conditions: clean, bandpass, two noise levels, and codec (below).
Bandpass (300–3400 Hz)	A filter that keeps only the narrow “telephone band” of frequencies, mimicking how radio and phone channels cut off very low and very high sounds.
AWGN (Additive White Gaussian Noise)	Adding random hiss to the audio to simulate a noisy channel.
SNR (Signal-to-Noise Ratio), 10 dB / 0 dB	How loud the speech is compared to the noise. 10 dB is moderately noisy; 0 dB means speech and noise are equally loud (very hard). SeamlessM4T's lead grows most at 0 dB.
MP3 codec (16 kbit/s)	Heavy audio compression that introduces distortion, simulating low-bandwidth transmission.
Diarization	Working out “who spoke when” — separating a conversation into individual speakers.
Denoise	Cleaning noise out of the audio. In the optional three-machine mode a dedicated model (DeepFilterNet3) does this.

4.6 Scripts, text and scoring

Diacritics	Small marks added to letters (accents, dots, vowel signs). Kashmiri's script uses many; both models tend to omit them, which distorts raw word-error scores.
Perso-Arabic / Nastaliq	The Arabic-derived script (in its Nastaliq style) used to write Urdu, Pashto and Kashmiri.
Devanagari / Gurmukhi / Han	The scripts for Hindi & Nepali (Devanagari), Punjabi (Gurmukhi), and Mandarin Chinese (Han characters).
CJK / character segmentation	Chinese-Japanese-Korean text has no spaces between words, so it must be scored per character. Missing this caused an early, wrong “100% error” result for Mandarin.
Whitespace tokenisation	Splitting text into “words” at spaces to score WER. It misleads on space-less scripts (Chinese) and diacritic-heavy scripts (Kashmiri) — a recurring source of scoring bugs.
Normalisation	Standardising text before scoring (e.g. removing diacritics, unifying character variants) so trivial spelling differences don't count as errors. Correcting the Kashmiri normaliser reversed the verdict there.

Scoring-methodology corrections (×5)	Five separate measurement mistakes the project found and fixed: a mislabelled baseline model, the Mandarin whitespace artefact, a translation label-encoding bug, an adapter-override bug in the test harness, and the Kashmiri diacritic-ruler issue. Each one had initially looked like a real model result.
---	--

4.7 System and deployment terms

Offline / air-gapped	The system runs with no internet connection at all — required for a secure/classified environment. All models are stored locally.
CTranslate2 (CT2) / int8 quantisation	A technique to run models faster and smaller by storing their numbers at 8-bit precision instead of full precision, with almost no accuracy loss — important on a small GPU.
GPU / VRAM / CPU parking	The graphics card (GPU) does the heavy computation; VRAM is its limited memory (8 GB here). “CPU parking” means moving idle models to ordinary RAM so several models fit within the small GPU memory.
Backend	The specific speech engine actually used for a language (e.g. “SeamlessM4T + LoRA” or “fine-tuned Whisper”).
Rollback	Keeping the old models on disk so the system can revert to them if needed, even though they are no longer the active choice.
3-node LAN mode	An optional distributed setup across three networked machines: one denoises and diarizes, one identifies language and dialect, and one (VANI) orchestrates everything.
5W	Who, What, When, Where, Why — the structure of the intelligence summary.
Coded-terminology lexicon	A dictionary of disguised words used in intercepts (e.g. an innocent word standing in for a weapon), so the threat detector can flag them.

This companion mirrors the current VANI deck and reports. If a slide or number changes, regenerate this document (generate_explainer_pdf.py) alongside the deck and the fine-tuning report.